Chapter 4
Induction and Recursion

MAD101

Ly Anh Duong

duongla3@fe.edu.vn

Table of Contents

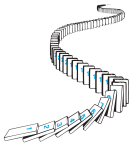
Principle of Mathematical Induction

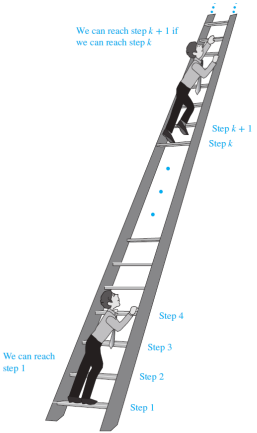
1 Mathematical Induction

To prove that $P(n)$ is true for all positive integers n, where $P(n)$ is a propositional function, we complete two steps:

Basis step. We verify that $P(1)$ is true.

Inductive step. We show that the conditional statement $P(k) \rightarrow P(k+1)$ is true for all positive integers k.





Example

1 Mathematical Induction

Example 1.1. Prove that $1 + 2 + 3 + ... + n - 1 + n = \dfrac{n(n+1)}{2}$.

Solution. Let $P(n)$: "$1 + 2 + \ldots + n = \dfrac{n(n+1)}{2}$".

Basis Step: $P(1)$: "$1 = \dfrac{1(1+1)}{2}$" is true.

Inductive Step:

Assume $P(k)$: "$1 + 2 + \ldots + k = \dfrac{k(k+1)}{2}$" is true with arbitrary $k > 0$.

We have

$$1 + 2 + \ldots + k + (k+1) = \frac{k(k+1)}{2} + (k+1) = \frac{(k+1)(k+2)}{2}$$

$\Longrightarrow P(k) \longrightarrow P(k+1)$ is true.

The statement is therefore proved by mathematical induction.

Example

1 Mathematical Induction

Example 1.4. Prove that

$$P(n) : \text{“} \sum_{j=0}^{n} ar^j = a + ar + ar^2 + \cdots + ar^n = \frac{ar^{n+1} - a}{r - 1}, \ (r \neq 1)\text{”}$$

where n is a nonnegative integer.

Solution.

Basis Step: $P(0) : \text{“} \sum_{j=0}^{0} ar^j = \frac{ar^{0+1}-a}{r-1} = \frac{ar-a}{r-1} = a\text{”}$ is true.

Inductive Step: The IH is the statement that $P(k) : \text{“} \sum_{j=0}^{k} ar^j = \frac{ar^{k+1} - a}{r - 1}\text{”}$ is true.

We have $\sum_{j=0}^{k+1} ar^j = \left(\sum_{j=0}^{k} ar^j\right) + ar^{k+1} = \frac{ar^{k+1}-a}{r-1} + ar^{k+1} = \frac{ar^{k+2}-a}{r-1}$

Thus, by mathematical induction, the formula is proven to be valid for all non-negative integers n.

Strong Induction

1 Mathematical Induction

To prove P(n) is true for all positive integers n, where P(n) is a propositional function, two steps are performed:
Basis step. Verifying $P(1)$ is true.
Inductivestep. Show $[P(1) \wedge P(2) \wedge ... \wedge P(k)] \rightarrow P(k+1)$ is true for all $k > 0$.

Example 1.5. Prove that if n is an integer greater than 1, then n can be written as the product of primes.

Example 1.6. Prove that every amount of postage of 12 cents or more can be formed using just 4-cents and 5-cents stamps.

Solution of Example 1.5

Let P(n): "n can be written as the product of primes".

Basis step:

$P(2) = 2$

$P(3) = 3$

$P(4) = 4 = 2.2$

...

Inductive step:

Suppose $P(j)$ is true $\forall j \leq k$.

Show $P(j)$ is true with $j = k + 1$:

i. If $k + 1$ is a prime, then $P(k + 1)$ is true.

ii. If $k + 1$ is a composite, then $k + 1 = ab, 2 \leq a \leq b < k + 1$. Because $a, b < k + 1$, according to hypothesis, a and b can be written as a product of primes. Hence, $k + 1$ can be written as a product of primes.

Solution of Example 1.6

Let $P(n)$: "n cents can be formed using just 4-cent and 5-cent stamps".

- **Basis step:**
  - $P(12)$ is true: $12 = 3.4$
  - $P(13)$ is true: $13 = 2.4 + 1.5$
  - $P(14)$ is true: $14 = 1.4 + 2.5$
  - $P(15)$ is true: $15 = 3.5$
  - .....
- **Inductive step:**
  Given $k \geq 16$ and suppose that $P(n)$ holds for all $n < k$. We now prove that $P(k)$ is also true.
  Obviously, $P(k-4)$ is true, i.e. $P(k-4)$ can be formed by some combination of 4-cent and 5-cent stamps. This implies to $P(k)$ by adding a 4-cent stamp.

Table of Contents

Example 2.1. **Fibonacci Numbers:**

$$1, 1, 2, 3, 5, 8, 13, 21, 34, 55, \ldots$$

n	1	2	3	4	5	6	7
F_n	1	1	2	3	5	8	13

1. Find F_8.
2. Find F_{16} if $F_{18} = 2584, F_{19} = 4181$.

Example 2.2. **Fibonacci Numbers:**

$$1, 1, 2, 3, 5, 8, 13, 21, 34, 55, \ldots$$

n	1	2	3	4	5	6	7
F_n	1	1	2	3	5	8	13

1. Find F_8.
2. Find F_{16} if $F_{18} = 2584, F_{19} = 4181$.

Solution. $F_8 = 21$; $F_{16} = 987$.

Example 2.3. **Fibonacci Numbers:**

$$1, 1, 2, 3, 5, 8, 13, 21, 34, 55, \ldots$$

n	1	2	3	4	5	6	7
F_n	1	1	2	3	5	8	13

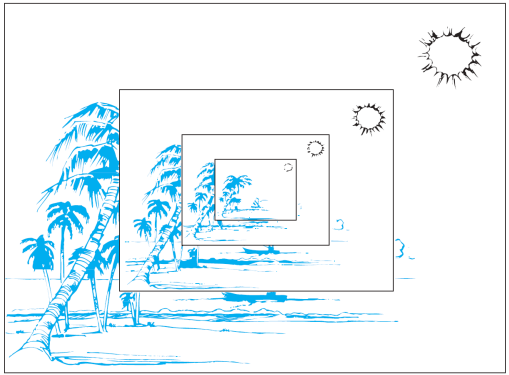
1. Find F_8.
2. Find F_{16} if $F_{18} = 2584, F_{19} = 4181$.

Solution. $F_8 = 21$; $F_{16} = 987$.
In general, this infinite sequence can be formulated by: $\boldsymbol{F_1 = F_2 = 1}$ and

$$\boldsymbol{F_n = F_{n-1} + F_{n-2} \quad \forall\, n \geq 3.}$$

Introduction

2 Recursive



Sometimes, it is difficult to define an object explicitly. However, it may be easy to define this object in terms of itself. This process is called **recursion**.

Recursive definition of Fibonacci numbers

Example 2.4.

Procedure Fibo(n: positive integer)
if $n = 1$ **or** $n = 2$
 return 1
else
 return Fibo$(n-1)$+Fibo$(n-2)$

What is output value if input $n = 5$?

- Basis step $F_1 = 1, F_2 = 1$
- Recursive step $F_n = F_{n-1} + F_{n-2}, n \geq 3$.

Recursively Defined Functions (hàm đệ quy)

2 Recursive

Definition 2.1

Two steps to define a function with the set of nonnegative integers as its domain:

- **Basis step.** Specify the value of the function at zero.
- **Recursive step.** Give a rule for finding its value at an integer from its values at smaller assessment integers.

Example 2.5. Give an algorithm to find pseudo-random numbers if $x_0 = 1, x_{n+1} = (3x_n + 17) \bmod 22$.

Solution.

Procedure pseudo(n:positive integer)
if $n = 0$
return 1
else
return (3*pseudo($n-1$)+17) mod 22

Example 2.6.

Procedure sum(n:$n \geq 1$, integer)
if $n = 1$
return 1
else
return sum($n-1$)+n

If input $n = 4$, what is the value of output?

Examples

2 Recursive

Example 2.7. Give the recursive definition of a^n (where a is a nonzero real number and n is a nonnegative integer).

Examples

2 Recursive

Example 2.9. Give the recursive definition of a^n (where a is a nonzero real number and n is a nonnegative integer).

Solution. Let $f(n) = a^n$. Then it yields

$$\boldsymbol{f(0) = 1,\ f(n) = af(n-1) \quad \forall n \geq 1.}$$

Example 2.10. Suppose that f is defined recursively by $f(0) = 3, f(n+1) = 2f(n) + 3$. Find $f(1), f(2), f(3)$ and $f(4)$.

Examples

2 Recursive

Example 2.11. Give the recursive definition of a^n (where a is a nonzero real number and n is a nonnegative integer).

Solution. Let $f(n) = a^n$. Then it yields

$$\boldsymbol{f(0) = 1,\ f(n) = af(n-1) \quad \forall n \geq 1.}$$

Example 2.12. Suppose that f is defined recursively by $f(0) = 3, f(n+1) = 2f(n) + 3$. Find $f(1), f(2), f(3)$ and $f(4)$.

Solution.

$f(1) = 2f(0) + 3 = 2.3 + 3 = 9$
$f(2) = 2f(1) + 3 = 2.9 + 3 = 21$
$f(3) = 2f(3) + 3 = 2.21 + 3 = 45$
$f(4) = 2f(3) + 3 = 2.45 + 3 = 93$

Examples

2 Recursive

Example 2.13. Give a recursive definition of $\sum_{k=0}^{n} a_k$.

Examples

2 Recursive

Example 2.14. Give a recursive definition of $\sum_{k=0}^{n} a_k$.

Solution.

The first part of the recursive definition is $\sum_{k=0}^{0} a_k = a_0$.

The second part is $\sum_{k=0}^{n+1} a_k = \left(\sum_{k=0}^{n} a_k\right) + a_{k+1}$.

Thus, we can set $f(n) = \sum_{k=0}^{n} a_k$ and obtain the following recursive relation

$$\boldsymbol{f(n+1) = f(n) + a_{n+1}} \quad \text{for any } n \geq 0.$$

Recursively Defined Sets (tập đệ quy)

2 Recursive

Definition 2.2

Recursive definitions of sets have two parts

- **Basis step.** An initial collection of elements is specified.
- **Recursive step.** Rules for forming new elements in the set from those already known to be in the set are provided.

Example 2.15. Consider the subset S of the set of integers recursively defined by

- **Basis step.** $3 \in S$
- **Recursive step.** If $x \in S$ and $y \in S$, then $x + y \in S$.

Solution. $S = \{3, 6, 9, 12, 15, 18, 21, \dots\}$

Examples

2 Recursive

Example 2.16. Find S if

a. 1 is in S, if x is in S then x + 1 and x + 2 are in S.
b. 1 is in S, if x is in S then x + 3 is in S.
c. 1, 2 are in S, if x is in S then x + 3 is in S.

Examples

2 Recursive

Example 2.18. Find S if

a. 1 is in S, if x is in S then x + 1 and x + 2 are in S.
b. 1 is in S, if x is in S then x + 3 is in S.
c. 1, 2 are in S, if x is in S then x + 3 is in S.

Solution.

a. $S = \{1, 2, 3, ...\}$
b. $S = \{1, 4, 7, 10, ...\}$
c. $S = \{1, 2, 4, 5, 7, 8, ...\}$

Example 2.19. Give a recursive definition of each of these sets.

a. $A = \{2, 5, 8, 11, 14, ...\}$.
b. $B = \{..., -5, -1, 3, 7, 11, ...\}$.
c. $C = \{3, 12, 48, 192, 768, ...\}$.

Solution.

a. $2 \in A, x \in A \rightarrow x + 3 \in A$.
b. $3 \in A, x \in A \rightarrow x+4 \in B \wedge x-4 \in B$.
c. $3 \in C, x \in C \rightarrow 4x \in C$.

The set $\sum^*$ of strings

2 Recursive

Definition 2.3

The set $\sum^*$ of strings over the fintite set (alphabet) $\sum$ is defined recursively by

- **BASIS STEP:** $\lambda \in \sum^*$ (where λ is the empty string containing no symbols).
- **RECURSIVE STEP:** If $w \in \sum^*$ and $x \in \sum$, then $wx \in \sum^*$.

Example 2.20.

1. $\sum = \{1\}$
   - $\sum_0^* = \{\lambda\}$
   - $\sum_1^* = \{\lambda, 1\}$
   - $\sum_2^* = \{\lambda, 1, 11\}$
   - $\sum_3^* = \{\lambda, 1, 11, 111\}$
   - $\sum_4^* = \{\lambda, 1, 11, 111, 1111\}$
   - $\ldots$

2. $\sum = \{0, 1\}$
   - $\sum_0^* = \{\lambda\}$
   - $\sum_1^* = \{\lambda, 0, 1\}$
   - $\sum_2^* = \{\lambda, 0, 1, 00, 01, 10, 11\}$
   - $\ldots$

Strings Concatenation (nối các xâu)

2 Recursive

Definition 2.4

Two strings can be combined via the operation of concatenation. Let $\sum$ be a set of symbols and $\sum^*$ the set of strings formed from symbols in $\sum$. We can define the concatenation of two strings, denoted by $\cdot$, recursively as follows.

- **BASIS STEP:** If $w \in \sum^*$, then $w \cdot \lambda = w$, where λ is the empty string.
- **RECURSIVE STEP:** If $w_1 \in \sum^*$ and $w_2 \in \sum^*$ and $x \in \sum$, then $w_1 \cdot (w_2 x) = (w_1 \cdot w_2)x$.

Example 2.21. The concatenation of $w_1 = abra$ and $w_2 = cadabra$ is $w_1 w_2 = abracadabra$.

Example 2.22. The length of a string can be recursively defined by

- $l(\lambda) = 0$
- $l(wx) = l(w) + 1$ if $w \in \sum^*$ and $x \in \sum$.

Table of Contents

3 Recursive Algorithms

Recursive Algorithms

3 Recursive Algorithms

Definition 3.1

An algorithm is called **recursive** if it solves a problem by reducing it to an instance of the same problem with smaller input.

Example 3.1. Using the algorithm to compute 5!

ALGORITHM 1 A Recursive Algorithm for Computing $n!$.

procedure *factorial*(n: nonnegative integer)
if $n = 0$ **then return** 1
else return $n \cdot factorial(n - 1)$
{output is $n!$}

Solution of Example 3.1

- $5! = 5.4!$
- $4! = 4.3!$
- $3! = 3.2!$
- $2! = 2.1!$
- $1! = 1.0!$
- $0! = 1$(Basis step)
- Recursive steps

  $1! = 1$
  $2! = 2$
  $3! = 6$
  $4! = 24$
  $5! = 120$

Recursive Algorithm for Computing a^n

3 Recursive Algorithms

ALGORITHM 2 A Recursive Algorithm for Computing a^n.

procedure *power*(a: nonzero real number, n: nonnegative integer)
if $n = 0$ **then return** 1
else return $a \cdot power(a, n-1)$
{output is a^n}

Example 3.2. Find output value if $a = 3, n = 4$.

Solution of Example 3.2

- $3^4 = 3.3^3$
- $3^3 = 3.3^2$
- $3^2 = 3.3^1$
- $3^1 = 3.3^0$
- $3^0 = 1$ (Basis step)
- Recursive step

  $3^1 = 3$
  $3^2 = 9$
  $3^3 = 27$
  $3^4 = 81$

Recursive Algorithm for Computing $\gcd(a, b)$

ALGORITHM 3 A Recursive Algorithm for Computing gcd(a, b).

procedure *gcd*(a, b: nonnegative integers with $a < b$)
if $a = 0$ **then return** b
else return *gcd*(b **mod** a, a)
{output is gcd(a, b)}

Example 3.3. Find output value if input $a = 5, b = 8$.

Solution of Example 3.3

- $\gcd(5, 8) = \gcd(8 \bmod 5, 5) = \gcd(3, 5)$
- $\gcd(3, 5) = \gcd(5 \bmod 3, 3) = \gcd(2, 3)$
- $\gcd(2, 3) = \gcd(3 \bmod 2, 2) = \gcd(1, 2)$
- $\gcd(1, 2) = \gcd(2 \bmod 1, 1) = \gcd(0, 1)$
- return 1

Hence, $\gcd(5, 8) = 1$.

Recursive Modular Exponentiation

ALGORITHM 4 Recursive Modular Exponentiation.

procedure *mpower*(b, n, m: integers with $b > 0$ and $m \geq 2$, $n \geq 0$)
if $n = 0$ **then**
    return 1
else if n is even **then**
    return $mpower(b, n/2, m)^2$ **mod** m
else
    return $(mpower(b, \lfloor n/2 \rfloor, m)^2$ **mod** $m \cdot b$ **mod** $m)$ **mod** m
{output is b^n **mod** m}

Example 3.4. Find 2^5 mod 3.

Solution of Example 3.4

$b = 2, n = 5, m = 3$

$n = 5$ odd: $mpower(2, 5, 3) = (mpower(2, 2, 3)^2 \bmod m.2 \bmod m) \bmod m$

$n = 2$ even: $mpower(2, 2, 3) = (mpower(2, 1, 3)^2) \bmod 3$

$n = 1$ odd: $mpower(2, 1, 3) = (mpower(2, 0, 3)^2 \bmod 3.2 \bmod 3) \bmod 3$

$mpower(2, 0, 3) = 1$ (Basis step)

Recursive steps

- $mpower(2, 1, 3) = 2$
- $mpower(2, 2, 3) = 1$
- $mpower(2, 5, 3) = 2$

Hence, $2^5 \bmod 3 = 2$

Recursive Linear Search Algorithm

3 Recursive Algorithms

ALGORITHM 5 A Recursive Linear Search Algorithm.

procedure *search*(i, j, x: integers, $1 \leq i \leq j \leq n$)
if $a_i = x$ **then**
    return i
else if $i = j$ **then**
    return 0
else
    return *search*($i + 1, j, x$)
{output is the location of x in $a_1, a_2, \ldots, a_n$ if it appears; otherwise it is 0}

Example 3.5. List all the steps used to search for 9 in the sequence $2, 3, 4, 5, 6, 8, 9, 11$.

Solution of Example 3.5

2	3	4	5	6	8	9	11
a_1	a_2	a_3	a_4	a_5	a_6	a_7	a_8

$\rightarrow i = 1, j = 8, x = 9$

- $a_1 = 9(!) \rightarrow$ Search$(2, 8, 9)$
- $a_2 = 9(!) \rightarrow$ Search$(3, 8, 9)$
- $a_3 = 9(!) \rightarrow$ Search$(4, 8, 9)$
- $a_4 = 9(!) \rightarrow$ Search$(5, 8, 9)$
- $a_5 = 9(!) \rightarrow$ Search$(6, 8, 9)$
- $a_6 = 9(!) \rightarrow$ Search$(7, 8, 9)$
- $a_7 = 9(ok)$
- return 7

Recursive Binary Search Algorithm

3 Recursive Algorithms

ALGORITHM 6 A Recursive Binary Search Algorithm.

procedure *binary search*$(i, j, x$: integers, $1 \le i \le j \le n)$
$m := \lfloor (i + j)/2 \rfloor$
if $x = a_m$ **then**
    return m
else if $(x < a_m$ and $i < m)$ **then**
    return *binary search*$(i, m - 1, x)$
else if $(x > a_m$ and $j > m)$ **then**
    return *binary search*$(m + 1, j, x)$
else return 0
{output is location of x in $a_1, a_2, \ldots, a_n$ if it appears; otherwise it is 0}

Example 3.6. To search for 19 in the list

$$1, 2, 3, 5, 6, 7, 8, 10, 12, 13, 15, 16, 18, 19, 20, 22$$

Solution of Example 3.6

1	2	3	5	6	7	8	10	12	13	15	16	18	19	20	22
a_1	a_2	a_3	a_4	a_5	a_6	a_7	a_8	a_9	a_{10}	a_{11}	a_{12}	a_{13}	a_{14}	a_{15}	a_{16}

$\rightarrow i = 1, j = 16, x = 19$

- $m := \lfloor (1 + 16)/2 \rfloor = 8 : 19 > 10 \wedge 16 > 8 \rightarrow$ binary search$(9, 16, 19)$
- $m := \lfloor (9 + 16)/2 \rfloor = 12 : 19 > 16 \wedge 16 > 12 \rightarrow$ binary search$(13, 16, 19)$
- $m := \lfloor (13 + 16)/2 \rfloor = 14 : 19 = 19$
- return 14

Recursive Algorithm for Fibonacci Numbers

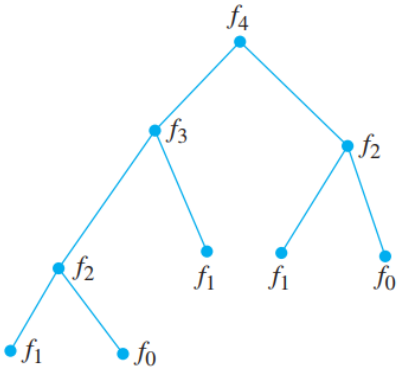
3 Recursive Algorithms

ALGORITHM 7 A Recursive Algorithm for Fibonacci Numbers.

procedure *fibonacci*(n: nonnegative integer)
if $n = 0$ **then return** 0
else if $n = 1$ **then return** 1
else return *fibonacci*$(n - 1)$ + *fibonacci*$(n - 2)$
{output is *fibonacci*(n)}

Example 3.7. How many additions (+) are used to find fibonacci(4) by the algorithm above?

Solution of Example 3.7



Iterative Algorithm for Fibonacci Numbers

ALGORITHM 8 An Iterative Algorithm for Computing Fibonacci Numbers.

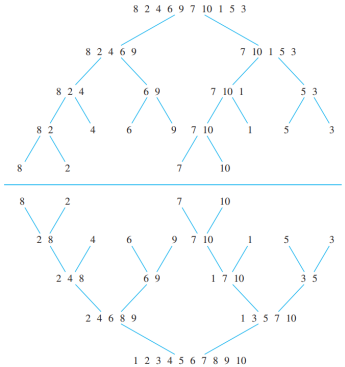
```
procedure iterative fibonacci(n: nonnegative integer)
if n = 0 then return 0
else
    x := 0
    y := 1
    for i := 1 to n − 1
        z := x + y
        x := y
        y := z
    return y
{output is the nth Fibonacci number}
```

Recursive Merge Sort

3 Recursive Algorithms

Example 3.9. Use the merge sort to put the terms of the list $8, 2, 4, 6, 9, 7, 10, 1, 5, 3$ in increasing order.

Solution.



Recursive Merge Sort

3 Recursive Algorithms

ALGORITHM 9 A Recursive Merge Sort.

procedure *mergesort*($L = a_1, \ldots, a_n$)
if $n > 1$ **then**
$m := \lfloor n/2 \rfloor$
$L_1 := a_1, a_2, \ldots, a_m$
$L_2 := a_{m+1}, a_{m+2}, \ldots, a_n$
$L := merge(mergesort(L_1), mergesort(L_2))$
{L is now sorted into elements in nondecreasing order}

Theorem 3.1

The number of comparisons needed to merge sort a list with n elements is $O(nlogn)$.

Example 3.10. Merging the Two Sorted Lists $2, 3, 5, 6$ and $1, 4$.

Solution of Example 3.10

TABLE 1 Merging the Two Sorted Lists 2, 3, 5, 6 and 1, 4.

First List	*Second List*	*Merged List*	*Comparison*
2 3 5 6	1 4		$1 < 2$
2 3 5 6	4	1	$2 < 4$
3 5 6	4	1 2	$3 < 4$
5 6	4	1 2 3	$4 < 5$
5 6		1 2 3 4	
		1 2 3 4 5 6	

Q&A

Thank you for listening!